Decision Tree

A **Decision Tree** is a supervised machine learning algorithm used for both:

- **Classification** → predicting categories (Fraud / Not Fraud)
- **Regression** → predicting numerical values (Insurance claim amount)

It works by splitting data into smaller subsets based on conditions until a decision can be made.

1. Decision Tree for Classification

Used when the output is **categorical**.

Example:

Predict whether an insurance claim is:

- Fraud
- Not Fraud

2. Decision Tree for Regression

Used when the output is **continuous numeric value**.

Example:

- Predict **insurance claim cost**.

3. Pure Node

A pure node means:

All data points in that node belong to the same class.

All are Fraud -- This is a pure node.

4. Impure Node

An impure node contains mixed classes.

The algorithm must split further to make the node pure.

Decision trees try to reduce impurity at every split.

Common impurity measures:

- **Gini Index**
- **Entropy**

5. Recursive Binary Splitting

Decision trees split data using Recursive Binary Splitting.

- The data is split into two groups at a time
- This process continues recursively

6. Greedy Approach

At each step the algorithm chooses the best split available at that moment.

It does not look ahead to future splits.

Tree Pruning (Trimming) in Decision Trees

Tree Pruning means removing unnecessary branches from a decision tree to make the model simpler and prevent overfitting, improve generalization.

Sometimes a decision tree grows too complex, learning noise from the training data. Pruning removes those non-useful branches.

1.Post Pruning (Backward Pruning)

Build the full decision tree first, then remove branches that do not improve model performance.

2. Pre Pruning (Forward Pruning)
Stop growing the tree before it becomes too complex.
Instead of building the full tree, the algorithm stops splitting when conditions are met.
3. Gain (Information Gain)
Gain means how much information is obtained after a split.
In decision trees, the algorithm selects the split that reduces impurity the most.
Information Gain measures how well a feature separates the data.
Information Gain = Parent impurity − Child impurity

Entropy
Entropy measures how random or impure the data is in a node.
Entropy tells us how mixed the classes are.
All same class → Entropy = 0 (pure node)
Classes are mixed → high entropy.
Decision trees try to **reduce entropy after splitting**.

Information Gain
Information Gain tells us how much entropy decreases after a split.
Information Gain helps the algorithm choose the best feature to split.

Gini Index:
Gini Index is a metric used in Decision Tree algorithms (especially CART) to measure how pure or impure a node is.
It helps the algorithm decide where to split the data.
Gini = 1-(0.4^2+0.6^2)
Gini = 0.48
0.4- Yes Class
0.6- No Class

0-Pure Node
Close to 0 – Low Impurity
Close to 0.5 - Highly mixed classes

Different Algorithms For Decison Tree:

1. ID3 (Iterative Dichotomiser 3)
ID3 builds a decision tree using Information Gain based on Entropy.
The feature with the highest information gain is chosen for splitting.
Characteristics

- Uses **Entropy**
- Works mainly for **categorical features**
- Used for **classification problems only**
- Does **not handle missing values**
- No pruning (can lead to overfitting)

ID3 chooses the feature that gives maximum information gain**.**

2. C4.5

C4.5 improves ID3 by solving its limitations.
This prevents bias toward attributes with many values.
Gain Ratio = Information Gain/split information

Characteristics

- Handles **continuous and categorical data**
- Handles **missing values**
- Uses **tree pruning**
- More accurate than ID3

Example

- If a feature like **Customer ID** has many unique values, ID3 may wrongly choose it.
  C4.5 avoids this by using **Gain Ratio**.

3. CART (Classification and Regression Trees)

CART uses Gini Index to measure impurity.
The algorithm chooses the split with lowest Gini impurity.

Characteristics

- Supports Classification and Regression
- Uses binary splits only (2 branches)
- Uses Gini Index
- Includes pruning
- Used in many ML libraries like Scikit-learn

Cross Validation

Cross Validation is a model evaluation technique where the dataset is split multiple times to test the model's performance.

Purpose:

- Reduce overfitting
- Ensure the model generalizes well
- Detect high variance models

1. **Hold-Out Method**
   The dataset is split **once** into training and testing data.
   - **80% Training data**
   - **20% Testing data**
   - Calculate performance metrics (Accuracy, F1, etc.)

Advantages

- **Very fast**
- **Simple to implement**

2. **K-Fold Cross Validation**
   The dataset is divided into **K equal parts (folds)**.
   If K = 5
   1. Split dataset into 5 folds
   2. Train on 4 folds
   3. Test on 1 fold
   4. Repeat until each fold becomes the test set

   So you get 5 model evaluations, then take the average accuracy.

Final performance = Average of all results

Hyperparameter Tuning

Hyperparameter tuning is the process of finding the best parameters of a model to improve its performance.

These parameters are set before training the model and are not learned from the data.

Goal:

- Improve **accuracy**
- Reduce **overfitting**
- Improve **generalization**

Grid Search

Grid Search is a technique used to find the best combination of hyperparameters.

It works by:

1. Defining possible values for parameters
2. Trying every possible combination
3. Evaluating performance using cross-validation
4. Selecting the best performing combination

Criterion in Decision Trees

1. Gini Index

Measures impurity of a node.

Lower value → better split

Used in CART algorithm

Faster to compute

2. **Entropy**

Measures information disorder in data.

- Used in **ID3 / C4.5**

Hyperparameter tuning → improves model performance

Grid Search → method to find best parameters

Criterion options in Decision Tree → Gini Index or Entropy

Inferencing (Inference)

Inference means **using a trained model to make predictions on new data**.

It happens **after the model training is completed**.

Process

1. Train the model on training data
2. Save the trained model
3. Use the model to predict results for **new unseen data**

Fine-Tuning

Fine-tuning means taking a pre-trained model and training it further on your own dataset to improve performance for a specific task.

Instead of training from scratch, we adjust the model weights slightly.

A model trained on **millions of images** (general object detection).

You fine-tune it with a dataset of:

- medical X-ray images
- insurance claim images
- retail product images

Steps

1. Start with **pre-trained model**
2. Add new dataset
3. Train a few layers or adjust weights
4. Model adapts to your task

Transfer Learning

Transfer Learning means reusing knowledge learned from one task and applying it to another related task.
Instead of building a model from scratch, we transfer knowledge from an existing model.

A model trained on **ImageNet (millions of images)** can be used for:

- face recognition
- medical imaging
- product classification

The model already understands:

- shapes
- edges
- textures

So we reuse that knowledge.

Ensemble Learning

Ensemble Learning combines several models (called weak learners) to create a stronger predictive model.
Goal:

- Improve accuracy
- Reduce overfitting
- Reduce variance and bias

1.Bagging (Bootstrap Aggregating)

Multiple models are trained **independently in parallel** on different random samples of the dataset.

Steps

1. Create multiple datasets using bootstrap sampling (random sampling with replacement).
2. Train separate models on each dataset.
3. Combine their predictions.

Final Prediction

- Classification: Majority voting
- Regression: Average of predictions

Famous Algorithm is Random Forest.

Advantages

- Reduces variance
- Handles overfitting

2.Boosting

Models are trained sequentially (one after another).
Each new model focuses on correcting errors made by previous models.

Steps

1. Train first model

2. Identify mistakes
3. Give higher importance to misclassified data
4. Train next model
5. Repeat

Final Prediction

Predictions are combined using weighted average.

Random Forest

- Random Forest is an ensemble learning algorithm that builds multiple decision trees and combines their predictions to produce a more accurate and stable result.
- It is based on the concept of Bagging (Bootstrap Aggregating).
- Instead of relying on **one decision tree**, Random Forest creates **many decision trees in parallel** and combines their outputs.
- Random Forest reduces the overfitting problem of a single decision tree.
  Final prediction:
  - Classification → Majority voting
  - Regression → Average of predictions

Step 1 — Bootstrap Sampling

Random subsets of the dataset are created using sampling with replacement.

Step 2 — Build Decision Trees

Each subset is used to train a separate decision tree.

But Random Forest adds another randomness:

At every split, the algorithm chooses from a random subset of features, not all features.

Features:

- Age
- Salary
- Credit score
-

Step 3 — Parallel Training

- All trees are trained **independently and simultaneously**.

Boosting Techniques:

Boosting algorithms are a family of **ensemble learning methods** where models are trained **sequentially**, and each new model focuses on correcting the errors of the previous one. Below are the main boosting techniques.

1. AdaBoost (Adaptive Boosting)

- Models are trained **sequentially**
- Misclassified samples get **higher weight**
- Correctly classified samples get **lower weight**

So the next model focuses more on **hard-to-classify data points**.

Key points

- Sequential learning
- Focuses on mistakes
- Final output = **weighted sum of models**

2. Gradient Boosting

- Gradient Boosting improves models by **reducing prediction errors using gradient descent**.

- Instead of changing data weights like AdaBoost, it **fits new models to the residual errors**.
- Each new model learns:
- Error = Actual − Prediction
- The next model tries to **correct this error**.

3.XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized version of Gradient Boosting designed for speed and performance.

Features:

- Regularization (reduces overfitting)
- Parallel processing
- Efficient handling of missing values
- Very popular in Kaggle competitions

4.**LightGBM**

LightGBM (Light Gradient Boosting Machine) is designed for large datasets and faster training.
Instead of growing trees level-by-level, it grows trees **leaf-wise**.

- Faster training
- Lower memory usage
- Works well with **big datasets**

5.CatBoost

- **CatBoost is designed especially for categorical data.**
- Automatically handles categorical variables
- Reduces overfitting
- Minimal data preprocessing required